Trainity Project 3

Operation Analytics and Investigating Metric Spike

Project Description: Analyze a company's end-to-end operations as a Lead Data Analyst, investigating metric spikes and deriving insights for various teams. Use advanced SQL to analyze datasets, identify trends, and explain sudden changes in key metrics like user engagement or sales to improve operational efficiency.

Case Study 1: Job Data Analysis

You will be working with a table named job_data with the following columns:

- job_id: Unique identifier of jobs
- actor_id: Unique identifier of actor
- event: The type of event (decision/skip/transfer).
- language: The Language of the content
- time_spent: Time spent to review the job in seconds.
- org: The Organization of the actor
- ds: The date in the format yyyy/mm/dd (stored as text).

Tasks:

A. Jobs Reviewed Over Time:
   - Objective: Calculate the number of jobs reviewed per hour for each day in November 2020.
   - Your Task: Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

B. Throughput Analysis:
   - Objective: Calculate the 7-day rolling average of throughput (number of events per second).
   - Your Task: Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why.

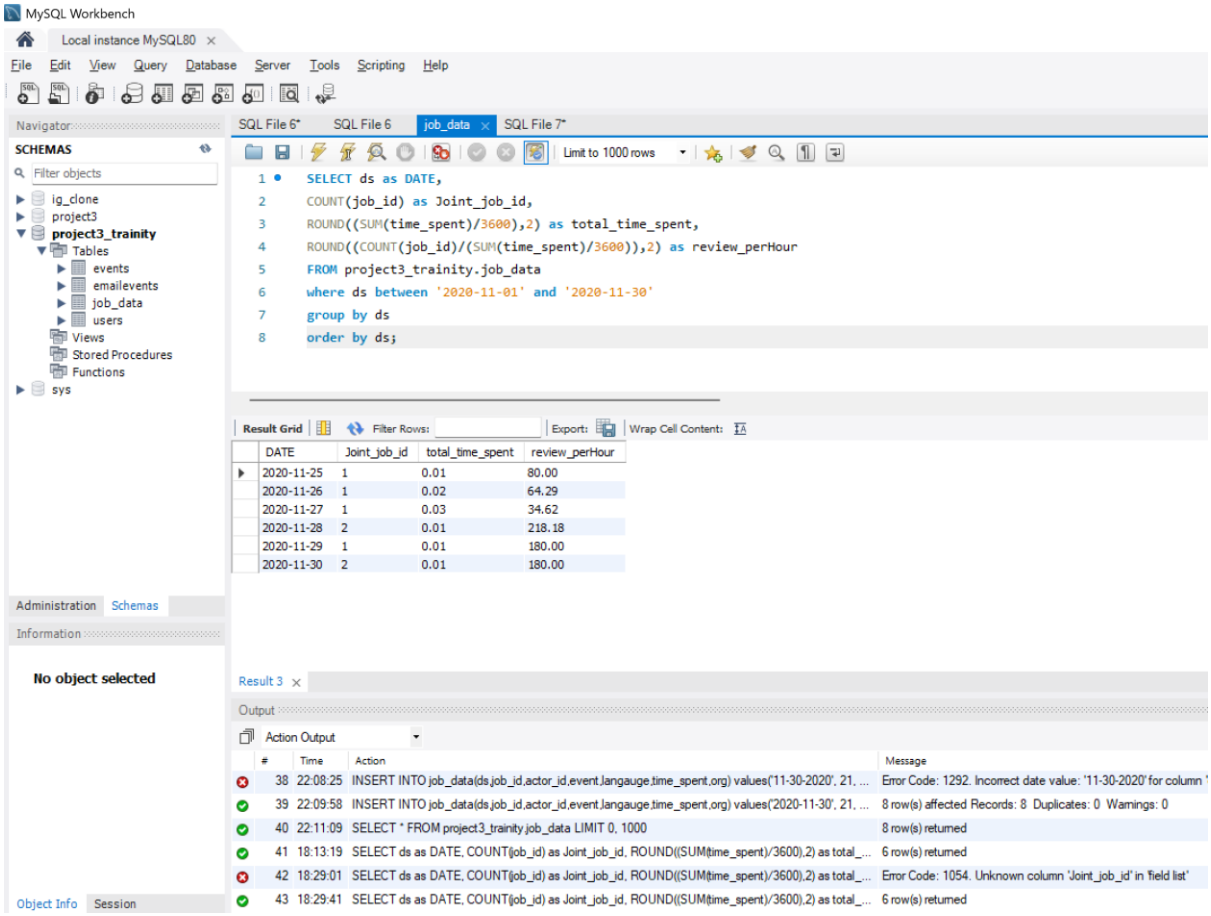
C. Language Share Analysis:
   - Objective: Calculate the percentage share of each language in the last 30 days.
   - Your Task: Write an SQL query to calculate the percentage share of each language over the last 30 days.

D. Duplicate Rows Detection:
   - Objective: Identify duplicate rows in the data.
   - Your Task: Write an SQL query to display duplicate rows from the job_data table.

Software Used: MySQL Workbench, Trainity Portal

Question 1: Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

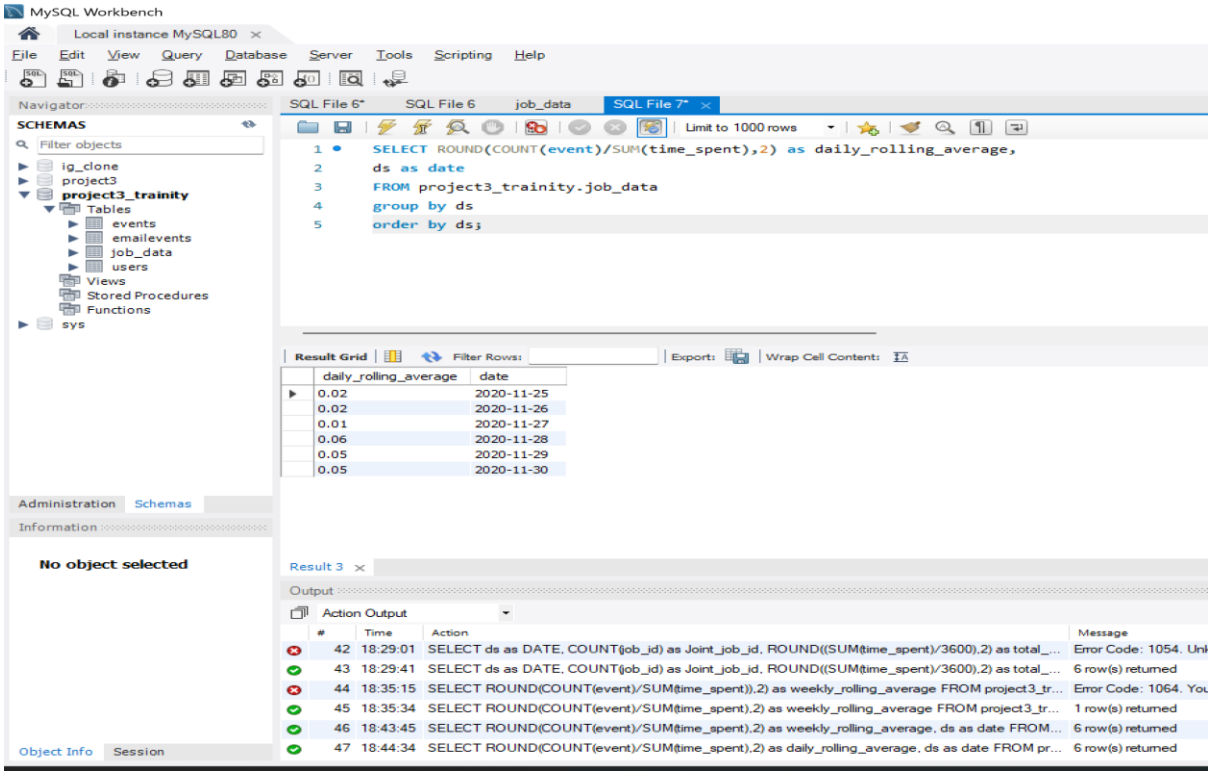
Ans:

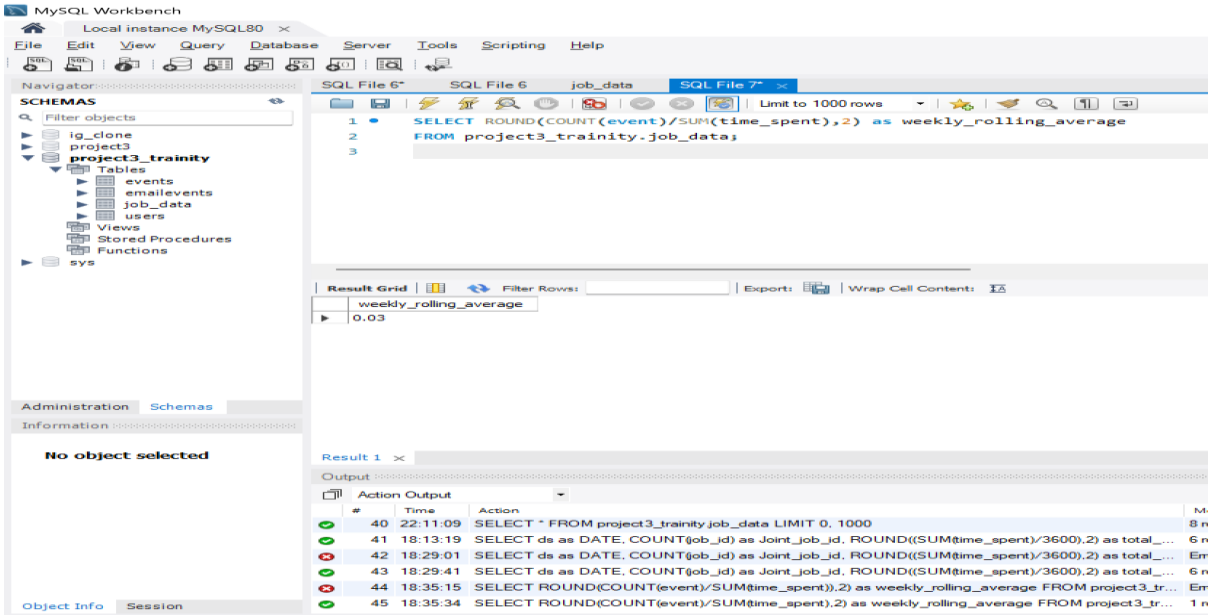


- The most reviews conducted per hour was on 2020-11-28 at 218.18 per hour and the least on 2020-11-27 at 34.62 per hour.

Question 2: Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why.

Ans:

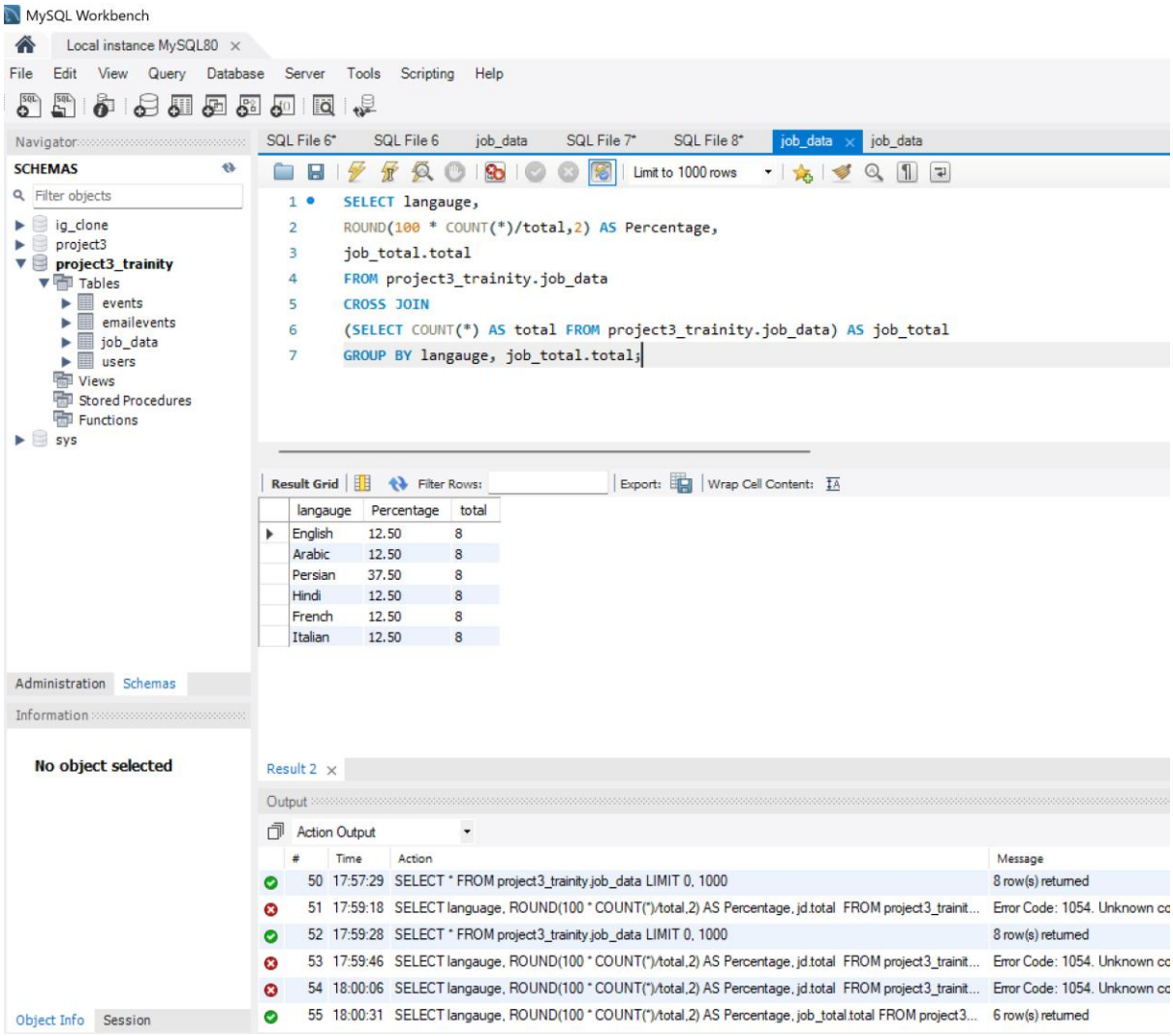




- The 7-day rolling average is preferred for trend analysis as it smooths daily fluctuations and provides a clearer view of throughput over time. It helps identify consistent patterns and long-term performance trends, reducing the impact of daily variations. However, if real-time monitoring or immediate issue detection is needed, the daily metric is more suitable.

Question 3: Write an SQL query to calculate the percentage share of each language over the last 30 days.

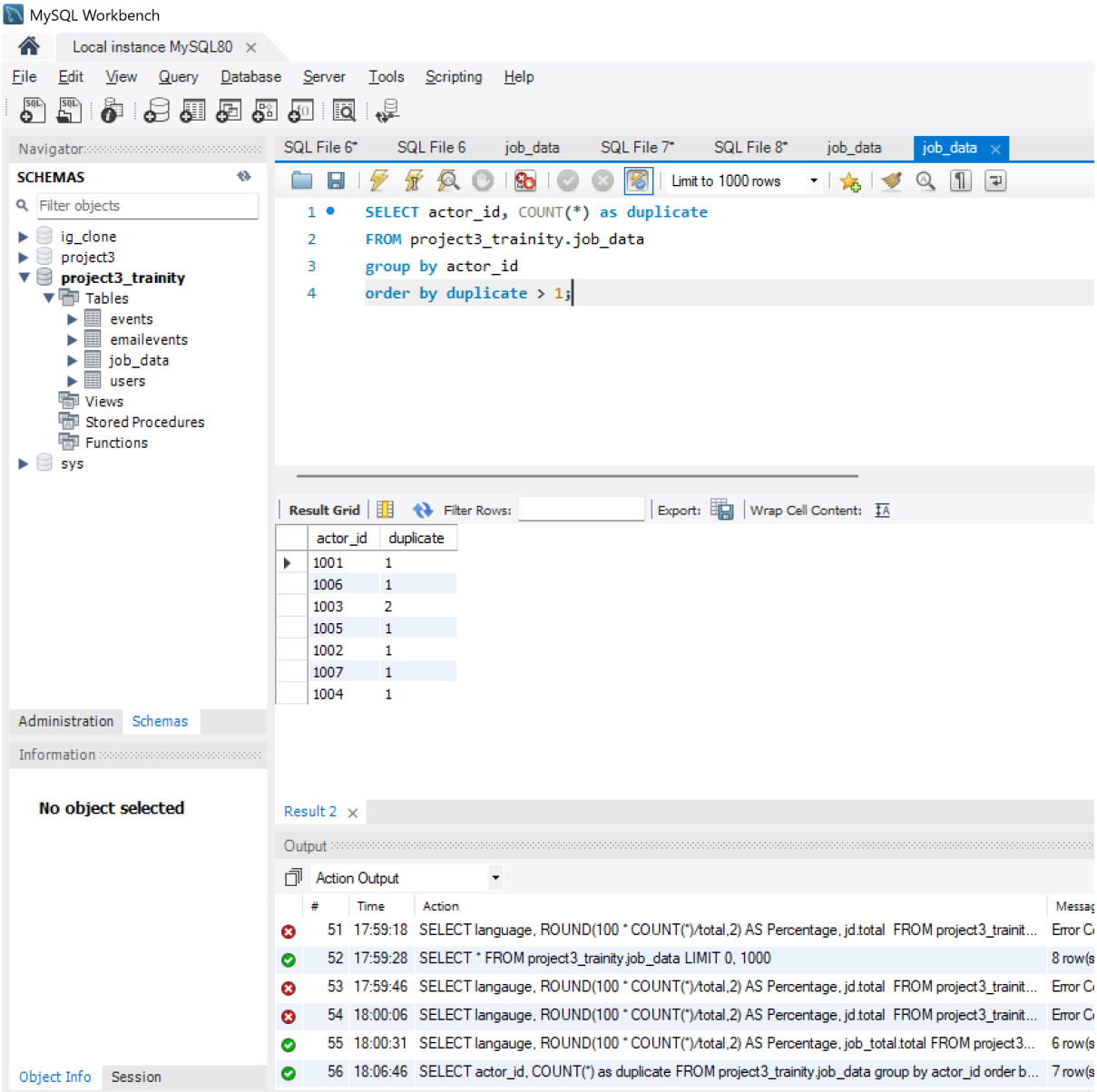
Ans:



- Persian language has the highest contribution of 37.50% whereas all the other languages contribute equally at 12.50%

Question 4: Write an SQL query to display duplicate rows from the job_data table.

Ans:



- Only one duplicate row is found within the data which is related to actor_id '1003'.

Case Study 2: Investigating Metric Spike

You will be working with three tables:

- **users**: Contains one row per user, with descriptive information about that user's account.
- **events**: Contains one row per event, where an event is an action that a user has taken (e.g., login, messaging, search).
- **email_events**: Contains events specific to the sending of emails.

Tasks:

A. **Weekly User Engagement:**
   - Objective: Measure the activeness of users on a weekly basis.
   - Your Task: Write an SQL query to calculate the weekly user engagement.

B. **User Growth Analysis:**
   - Objective: Analyze the growth of users over time for a product.
   - Your Task: Write an SQL query to calculate the user growth for the product.

C. **Weekly Retention Analysis:**
   - Objective: Analyze the retention of users on a weekly basis after signing up for a product.
   - Your Task: Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

D. **Weekly Engagement Per Device:**
   - Objective: Measure the activeness of users on a weekly basis per device.
   - Your Task: Write an SQL query to calculate the weekly engagement per device.

E. **Email Engagement Analysis:**
   - Objective: Analyze how users are engaging with the email service.
   - Your Task: Write an SQL query to calculate the email engagement metrics.

Question 1: Write an SQL query to calculate the weekly user engagement.

Ans:

MySQL Workbench

```
SELECT EXTRACT(WEEK FROM occured_at) as week,
COUNT(DISTINCT user_id) as active_user
from
project3_trainity.events
where event_type = 'engagement'
group by week
order by week
```

week	active_user
17	663
18	1068
19	1113
20	1154
21	1121
22	1186
23	1232
24	1275
25	1264
26	1302
27	1372
28	1365
29	1376
30	1467
31	1299
32	1225
33	1225
34	1204
35	104

- Week '30' had the highest number of active users at 1467 whereas week '35' contributed the least with a total number of active users at 104.

Question 2: Write an SQL query to calculate the user growth for the product.

Ans:

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- ig_clone
- project3
- **project3_trainity**
  - Tables
    - events
    - emailevents
    - job_data
    - users
  - Views
  - Stored Procedures
  - Functions
- sys

Administration Schemas

Information

No object selected

SQL File 9* SQL File 10* SQL File 11*

Limit to 1000 rows

```
WITH active_users_per_week as (
SELECT EXTRACT(year from created_at) as year,
EXTRACT(week from created_at) as week,
COUNT(DISTINCT user_id) as total_users
from project3_trainity.users
GROUP BY year, week )

SELECT year, week, total_users,
SUM(total_users) OVER (ORDER BY year, week) as growth_rate
from active_users_per_week
order by year, week;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	week	total_users	growth_rate
2013	0	23	23
2013	1	30	53
2013	2	48	101
2013	3	36	137
2013	4	30	167
2013	5	48	215
2013	6	38	253
2013	7	42	295
2013	8	34	329
2013	9	43	372
2013	10	32	404
2013	11	31	435
2013	12	33	468
2013	13	39	507
2013	14	35	542
2013	15	43	585
2013	16	46	631
2013	17	49	680
2013	18	44	724
2013	19	57	781

Result 1

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- ig_clone
- project3
- **project3_trainity**
  - Tables
    - events
    - emailevents
    - job_data
    - users
  - Views
  - Stored Procedures
  - Functions
- sys

Administration Schemas

Information

No object selected

SQL File 9* SQL File 10* SQL File 11*

Limit to 1000 rows

```
WITH active_users_per_week as (
SELECT EXTRACT(year from created_at) as year,
EXTRACT(week from created_at) as week,
COUNT(DISTINCT user_id) as total_users
from project3_trainity.users
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	week	total_users	growth_rate
2013	20	39	820
2013	21	49	869
2013	22	54	923
2013	23	50	973
2013	24	45	1018
2013	25	57	1075
2013	26	56	1131
2013	27	52	1183
2013	28	72	1255
2013	29	67	1322
2013	30	67	1389
2013	31	67	1456
2013	32	71	1527
2013	33	73	1600
2013	34	78	1678
2013	35	63	1741
2013	36	72	1813
2013	37	85	1898
2013	38	90	1988
2013	39	84	2072
2013	40	87	2159
2013	41	73	2232
2013	42	99	2331
2013	43	89	2420
2013	44	96	2516
2013	45	91	2607
2013	46	88	2695
2013	47	102	2797

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- ig_clone
- project3
- **project3_trainity**
  - Tables
    - events
    - emailevents
    - job_data
    - users
  - Views
  - Stored Procedures
  - Functions
- sys

Administration Schemas

Information

No object selected

SQL File 9* | SQL File 10* | SQL File 11*

Limit to 1000 rows

```
WITH active_users_per_week as (
SELECT EXTRACT(year from created_at) as year,
EXTRACT(week from created_at) as week,
COUNT(DISTINCT user_id) as total_users
from project3_trainity.users
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	week	total_users	growth_rate
2013	46	88	2695
2013	47	102	2797
2013	48	97	2894
2013	49	116	3010
2013	50	124	3134
2013	51	102	3236
2013	52	47	3283
2014	0	83	3366
2014	1	126	3492
2014	2	109	3601
2014	3	113	3714
2014	4	130	3844
2014	5	133	3977
2014	6	135	4112
2014	7	125	4237
2014	8	129	4366
2014	9	133	4499
2014	10	154	4653
2014	11	130	4783
2014	12	148	4931
2014	13	167	5098
2014	14	162	5260
2014	15	164	5424
2014	16	179	5603
2014	17	170	5773
2014	18	163	5936
2014	19	185	6121
2014	20	176	6297

2894

Result 1

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- ig_clone
- project3
- **project3_trainity**
  - Tables
    - events
    - emailevents
    - job_data
    - users
  - Views
  - Stored Procedures
  - Functions
- sys

Administration Schemas

Information

No object selected

SQL File 9* | SQL File 10* | SQL File 11*

Limit to 1000 rows

```
WITH active_users_per_week as (
SELECT EXTRACT(year from created_at) as year,
EXTRACT(week from created_at) as week,
COUNT(DISTINCT user_id) as total_users
from project3_trainity.users
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	week	total_users	growth_rate
2014	9	133	4499
2014	10	154	4653
2014	11	130	4783
2014	12	148	4931
2014	13	167	5098
2014	14	162	5260
2014	15	164	5424
2014	16	179	5603
2014	17	170	5773
2014	18	163	5936
2014	19	185	6121
2014	20	176	6297
2014	21	183	6480
2014	22	196	6676
2014	23	196	6872
2014	24	229	7101
2014	25	207	7308
2014	26	201	7509
2014	27	222	7731
2014	28	215	7946
2014	29	221	8167
2014	30	238	8405
2014	31	193	8598
2014	32	245	8843
2014	33	261	9104
2014	34	259	9363
2014	35	18	9381

2894

Question 3: Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

Ans:

```sql
SELECT first as week_nums,
SUM(CASE when week_num = 0 then 1 else 0 end) as 'week_0',
SUM(CASE when week_num = 1 then 1 else 0 end) as 'week_1',
SUM(CASE when week_num = 2 then 1 else 0 end) as 'week_2',
SUM(CASE when week_num = 3 then 1 else 0 end) as 'week_3',
SUM(CASE when week_num = 4 then 1 else 0 end) as 'week_4',
SUM(CASE when week_num = 5 then 1 else 0 end) as 'week_5',
SUM(CASE when week_num = 6 then 1 else 0 end) as 'week_6',
SUM(CASE when week_num = 7 then 1 else 0 end) as 'week_7',
SUM(CASE when week_num = 8 then 1 else 0 end) as 'week_8',
SUM(CASE when week_num = 9 then 1 else 0 end) as 'week_9',
SUM(CASE when week_num = 10 then 1 else 0 end) as 'week_10',
SUM(CASE when week_num = 11 then 1 else 0 end) as 'week_11',
SUM(CASE when week_num = 12 then 1 else 0 end) as 'week_12',
SUM(CASE when week_num = 13 then 1 else 0 end) as 'week_13',
SUM(CASE when week_num = 14 then 1 else 0 end) as 'week_14',
SUM(CASE when week_num = 15 then 1 else 0 end) as 'week_15',
SUM(CASE when week_num = 16 then 1 else 0 end) as 'week_16',
SUM(CASE when week_num = 17 then 1 else 0 end) as 'week_17',
SUM(CASE when week_num = 18 then 1 else 0 end) as 'week_18'
from (
SELECT l.user_id, l.login_week, f.first, l.login_week - f.first as week_num
FROM ( SELECT user_id, EXTRACT(WEEK FROM occured_at) as login_week
FROM project3_trainity.events
group by user_id, login_week
) as l
JOIN ( SELECT user_id, MIN(EXTRACT( week from occured_at)) as first
from project3_trainity.events
group by user_id
) as f
on l.user_id = f.user_id ) as sub
group by first
order by first;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

week_nums	week_0	week_1	week_2	week_3	week_4	week_5	week_6	week_7	week_8	week_9	week_10	week_11	week_12	week_13	week_14	week_15	week_16	week_17	week_18
17	663	472	324	251	205	187	167	146	145	145	136	131	132	143	116	91	82	77	5
18	596	362	261	203	168	147	144	127	113	122	106	118	127	110	97	85	67	4	0
19	427	284	173	153	114	95	91	81	95	82	68	65	63	42	51	49	2	0	0
20	358	223	165	121	91	72	63	67	63	65	67	41	40	33	40	0	0	0	0
21	317	187	131	91	74	63	75	72	58	48	45	39	35	28	2	0	0	0	0
22	326	224	150	107	87	73	63	60	55	48	41	39	31	1	0	0	0	0	0
23	328	219	138	101	90	79	69	61	54	47	35	30	0	0	0	0	0	0	0
24	339	205	143	102	81	63	65	61	38	39	29	0	0	0	0	0	0	0	0
25	305	218	139	101	75	63	50	46	38	35	2	0	0	0	0	0	0	0	0
26	288	181	114	83	73	55	47	43	29	0	0	0	0	0	0	0	0	0	0
27	292	199	121	106	68	53	40	36	1	0	0	0	0	0	0	0	0	0	0
28	274	194	114	69	46	30	28	3	0	0	0	0	0	0	0	0	0	0	0
29	270	186	102	65	47	40	1	0	0	0	0	0	0	0	0	0	0	0	0
30	294	202	121	78	53	3	0	0	0	0	0	0	0	0	0	0	0	0	0
31	215	145	76	57	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
32	267	188	94	8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
33	286	202	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
34	279	44	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
35	18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Question 4: Write an SQL query to calculate the weekly engagement per device.

Ans:

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13*

Limit to 1000 rows

```
SELECT EXTRACT(week from occured_at) as week,
COUNT(DISTINCT CASE when device = 'acer aspire desktop' then user_id else null end) as acer_aspire_desktop,
COUNT(DISTINCT CASE when device = 'acer aspire notebook' then user_id else null end) as acer_aspire_notebook,
COUNT(DISTINCT CASE when device = 'amazon fire phone' then user_id else null end) as amazon_fire_phone,
COUNT(DISTINCT CASE when device = 'asus chromebook' then user_id else null end) as asus_chromebook,
COUNT(DISTINCT CASE when device = 'dell inspiron notebook' then user_id else null end) as dell_inspiron_notebook,
COUNT(DISTINCT CASE when device = 'dell inspiron desktop' then user_id else null end) as dell_inspiron_desktop,
COUNT(DISTINCT CASE when device = 'hp pavilion desktop' then user_id else null end) as hp_pavilion_desktop,
COUNT(DISTINCT CASE when device = 'htc one' then user_id else null end) as htc_one,
COUNT(DISTINCT CASE when device = 'ipad air' then user_id else null end) as ipad_air,
COUNT(DISTINCT CASE when device = 'ipad mini' then user_id else null end) as ipad_mini,
COUNT(DISTINCT CASE when device = 'iphone 4s' then user_id else null end) as iphone_4s,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

week	acer_aspire_desktop	acer_aspire_notebook	amazon_fire_phone	asus_chromebook	dell_inspiron_notebook	dell_inspiron_desktop	hp_pavilion_desktop	htc_one	ipad_air	ipad_mini	iphone_4s	iphone_5	iphone_5s	kindle_fire	lenovo_thinkpad
17	9	20	4	21	46	18	14	16	27	19	21	65	42	6	86
18	26	33	9	42	77	58	37	19	52	30	46	113	73	27	153
19	23	41	12	27	83	36	40	30	55	36	44	115	79	21	178
20	23	40	11	41	84	52	30	29	59	32	55	125	79	23	173
21	29	47	5	38	80	41	44	21	51	23	45	137	74	30	167
22	25	41	5	52	92	52	38	24	58	34	45	125	71	21	176
23	22	43	16	49	103	53	54	20	41	33	53	152	79	25	176
24	24	40	11	43	99	59	56	20	57	39	53	142	79	25	165
25	28	47	13	38	105	52	52	21	57	30	40	137	78	24	197
26	29	35	13	49	89	60	46	23	56	43	50	152	94	26	192
27	29	49	10	52	89	53	56	27	55	35	67	163	83	25	202
28	30	49	6	50	103	56	56	26	54	35	61	151	93	31	220
29	28	53	12	49	113	54	58	31	52	34	60	144	90	37	209
30	33	60	12	56	127	54	42	31	70	35	65	152	103	25	206
31	31	55	14	56	113	44	51	13	55	27	56	135	71	14	207
32	35	55	12	62	104	57	51	18	48	30	34	119	67	12	179
33	39	46	14	49	110	37	38	19	40	28	35	110	65	14	191
34	30	63	11	47	105	49	36	25	39	25	50	101	70	13	193
35	1	3	0	6	9	1	1	2	0	2	6	2	3	3	16

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13*

Limit to 1000 rows

```
COUNT(DISTINCT CASE when device = 'nexus 10' then user_id else null end) as nexus_10,
COUNT(DISTINCT CASE when device = 'nexus 7' then user_id else null end) as nexus_7,
COUNT(DISTINCT CASE when device = 'nexus 5' then user_id else null end) as nexus_5,
COUNT(DISTINCT CASE when device = 'nokia lumia 635' then user_id else null end) as nokia_lumia_365,
COUNT(DISTINCT CASE when device = 'samsumg galaxy tablet' then user_id else null end) as samsung_galaxy_tablet,
COUNT(DISTINCT CASE when device = 'samsung galaxy note' then user_id else null end) as samsung_galaxy_note,
COUNT(DISTINCT CASE when device = 'samsung galaxy s4' then user_id else null end) as samsung_galaxy_s4,
COUNT(DISTINCT CASE when device = 'windows surface' then user_id else null end) as windows_surface
FROM project3_trainity.events
where event_type = 'engagement'
group by week
order by week;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

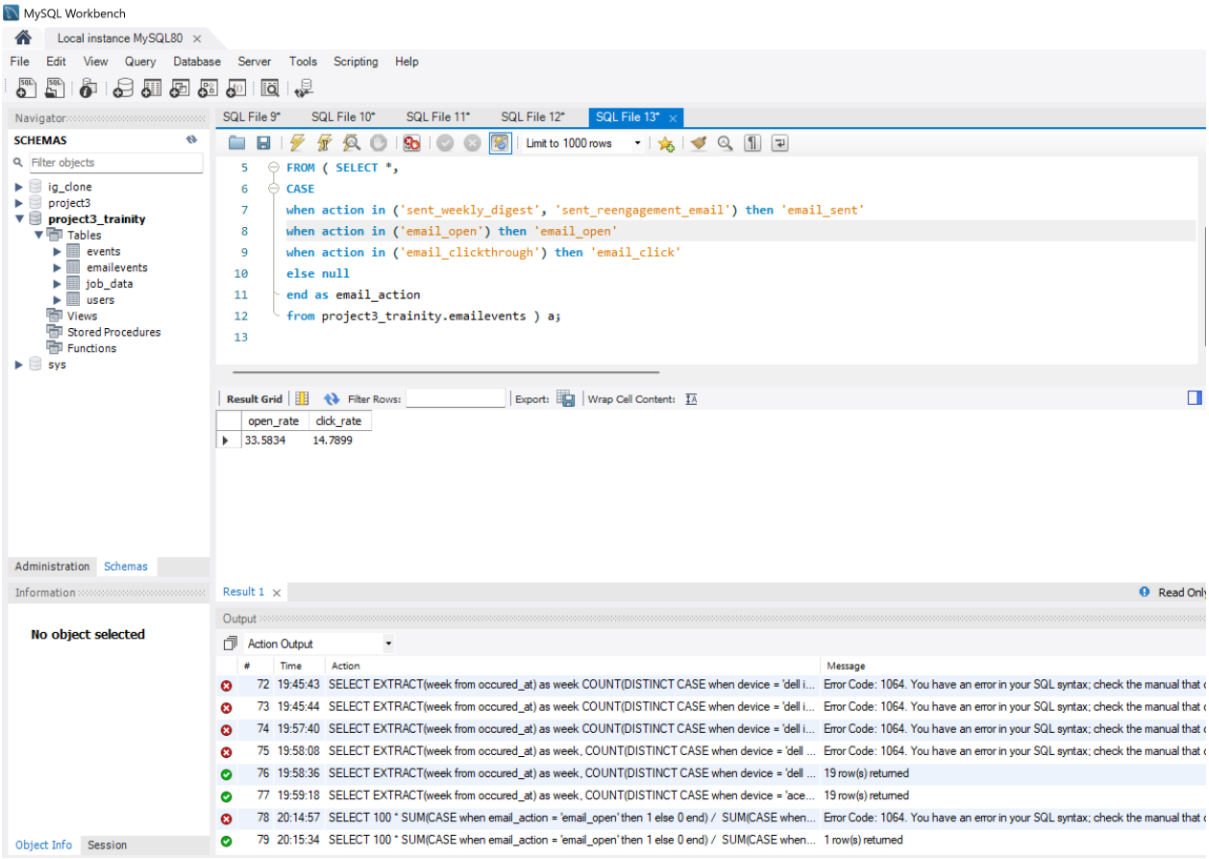
ir	ipad_mini	iphone_4s	iphone_5	iphone_5s	kindle_fire	lenovo_thinkpad	mac_mini	macbook_air	macbook_pro	nexus_10	nexus_7	nexus_5	nokia_lumia_365	samsung_galaxy_tablet	samsung_galaxy_note	samsung_galaxy_s4	windows_surface
19	21	65	42	6	86	6	54	143	16	18	40	17	8	7	52	10	
30	46	113	73	27	153	13	121	252	30	30	73	33	11	15	82	10	
36	44	115	79	21	178	18	112	266	25	41	87	23	6	11	91	16	
32	55	125	79	23	173	26	119	256	22	32	103	22	9	18	93	21	
23	45	137	74	30	167	18	110	247	25	29	91	25	6	20	84	17	
34	45	125	71	21	176	25	145	251	27	45	96	25	10	19	105	15	
33	53	152	79	25	176	18	124	266	45	36	88	31	14	14	99	14	
39	53	142	79	25	165	29	152	255	38	49	87	35	11	20	101	22	
30	40	137	78	24	197	21	121	275	29	51	89	37	12	14	99	22	
43	50	152	94	26	192	11	134	269	29	46	87	42	12	9	112	21	
35	67	163	83	25	202	15	142	302	37	40	84	31	15	15	116	33	
35	61	151	93	31	220	28	148	295	26	39	85	35	9	10	122	33	
34	60	144	90	37	209	31	148	295	25	45	77	43	13	16	123	28	
35	65	152	103	25	206	23	159	322	36	62	84	34	9	15	103	19	
27	56	135	71	14	207	24	147	321	24	38	69	28	8	14	100	19	
30	34	119	67	12	179	20	125	307	30	25	67	28	6	12	82	10	
28	35	110	65	14	191	32	133	312	23	30	70	27	12	13	80	15	
25	50	101	70	13	193	30	136	292	25	33	70	17	14	13	90	18	
2	6	2	3	3	16	2	10	17	2	2	4	2	0	1	6	3	

Result 2 | Read Only

- The weekly engagement for devices ‘Macbook Pro’, ‘Macbook Air’, ‘Lenovo Thinkpad’ and ‘Iphone 5’ was the highest.

Question 5: Write an SQL query to calculate the email engagement metrics.

Ans:



- The email open rate is at 33.58 whereas the email click rate is at 14.78.

Result: Through this project I understood the concept of sub-queries thoroughly, learnt about the 'EXTRACT' statement along with aggregate functions for calculations. I found this project really helping me with the concepts of 'CASE' statements alongside using 'WITH' statements to create specific custom tables. I plan on increasing my knowledge further on working with data related to time as I found that a little difficult here.